

Chaining in CeTTa

Goals, patterns, proofs and evidence

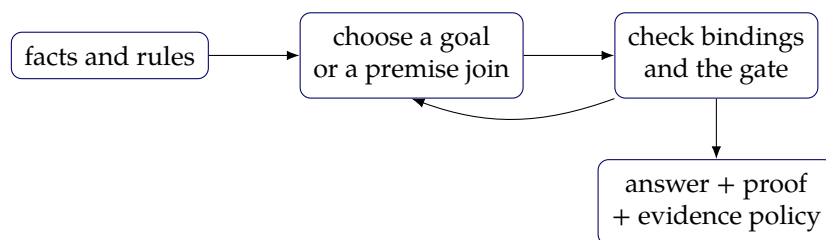
A runnable guide for HE and PeTTa programs

7 October 2026

A chainer connects facts to questions. A useful chainer also tells us which facts supported its answer, how much search it was allowed to perform, and whether an unanswered question was disproved or merely not reached.

This guide builds a small proof-producing backward chainer, a forward closure, and programmable search gates. It then adds directional matching, joint constraints, loop tests and occurrence-sensitive evidence. The final chapters connect these pieces to Nil Geisweiller's proof-size search and backward-via-forward experiments, and to the existing PLN examples.

The examples use CeTTa's HE and PeTTa dialects. Their common notation is useful, but the dialects are not interchangeable. Differences that affect a program are marked explicitly. The pat operations are CeTTa extensions; importing this module into another implementation does not install a native matcher there.



Contents

1 Quick start	3
2 Start with a question and a proof	3
3 Backward chaining with an explicit gate	4
4 Binding direction is independent of search direction	5
5 Forward chaining as controlled closure	7
6 Programmable gates without accidental guessing	8
7 Joint constraints and the scope of protection	9
8 Loop tests, memoization and tabling	10
9 Evidence you can inspect	12
9.1 Forward and backward clause subsumption	12
9.2 PLN: choose premises before combining truth values	13
10 Nil's proof-size search and backward-via-forward search	14
10.1 Changing the representation of unfinished work	15
10.2 A practical combination	15
11 Rules that rewrite without narrowing	15
12 Costs and limits	16
13 Exercises with counterexamples	17
A Reference card	17
B Reproducing the checks and measurements	18
B.1 A measured example	19
Sources and companion programs	19

1 Quick start

Build. CeTTa is a C program. On Debian or Ubuntu, install the build dependencies and build the default binary from the repository root:

```
sudo apt install build-essential pkg-config python3-dev libgmp-dev
make
```

This produces `./cetta`. GMP provides exact big integers; build with `make ENABLE_GMP=0` to do without it. SWI-Prolog is detected automatically when present and is optional. The repository's README lists the other build modes.

Run. All commands below run from the repository root. Every example names its dialect and profile:

```
./cetta --lang he --profile extended examples/chaining/tutorial/01-backward.metta
./cetta --lang petta --profile extended examples/chaining/tutorial/01-backward.metta
```

The `pat` operations and `=%` rules belong to the extended profiles. Their names stay ordinary data in other profiles. This backward-chaining example also runs in plain PeTTa.

Check. One command runs the tutorial and its companion SKI, clause and PLN clients, and compares their answers, negative cases, variable aliasing and evidence occurrences with the expected results:

```
make BUILD=python test-chaining-tutorial
```

Read the output. HE prints the answers of each `!` expression as one bracketed list; PeTTa prints each answer on its own line. `collapse` gathers all alternatives into one expression. HE's `[()]` after a collapsed failed search is therefore one empty collection, not an absence of answers. Import and include forms also print their dialect's setup result; the discussion below concerns the queries that follow them.

Appendix [A](#) is a one-page reference card for the matching operations.

2 Start with a question and a proof

Our first knowledge base is deliberately small:

`examples/chaining/tutorial/graph-facts.metta`

```
(fact ab (edge alice bob))
(fact bc (edge bob carol))
(fact cd (edge carol dana))
```

```
(rule direct ((edge $x $y)) (reachable $x $y))
(rule step ((edge $x $y) (reachable $y $z)) (reachable $x $z))
```

fact, rule and proof are ordinary data constructors chosen by this program. They are not new language primitives. Each fact has a label; each rule has a list of premises and a conclusion.

The question (reachable alice dana) needs three edges. An answer such as True would hide that reasoning. Instead, a leaf proof records its fact label, while a rule proof records its rule name and child proofs. For instance:

```
(proof (direct ((proof ab (edge alice bob))))
      (reachable alice bob))
```

A consumer can walk this tree and check each rule against the same knowledge base. Merely printing a value called proof is not independent proof checking: the producer and any checker still need a stated rule semantics.

3 Backward chaining with an explicit gate

Backward search starts with a desired conclusion. It tries a matching fact, then rules whose conclusions can unify with that goal. For a rule, it solves all premises in one shared environment. This is why the middle vertex of a path is the same in both premises.

examples/chaining/tutorial/backward-core.metta

```
; Proof-producing Horn search. The gate receives a remaining depth and a goal.
; Rule variables are freshly scoped by ordinary match on each activation.
(= (depth-gate $remaining $goal) (> $remaining 0))

(= (bc $gate $remaining $goal)
   (if ($gate $remaining $goal)
       (superpose
        ((match &self (fact $id $goal)
         (if ($gate $remaining $goal) (proof $id $goal) (empty)))
        (match &self (rule $name $premises $goal)
         (let $children (bc-all $gate (- $remaining 1) $premises)
          (if ($gate $remaining $goal)
              (proof ($name $children) $goal)
              (empty))))))
       (empty)))

(= (bc-all $gate $remaining $goals)
   (if (== $goals ())
       ()
       (let* (($first (bc $gate $remaining (car-atom $goals)))
              ($rest (bc-all $gate $remaining (cdr-atom $goals))))
         (cons-atom $first $rest))))
```

Read bc as follows. First ask the gate whether this search is allowed. Then choose a fact or a rule. A fact returns a leaf proof. A rule recursively proves its premises and returns a node whose

children are those proofs. `bc-all` threads the ordinary bindings through the premise list using `let*`. A failed premise produces no proof for that rule application.

The gate is checked again after a fact is selected or the premises are solved. This matters: a goal such as `(edge bob $to)` is less informative than the resulting `(edge bob carol)`. A policy that rejects the latter cannot rely solely on the earlier open goal.

Ordinary match is intentional here. A query with an unknown destination should discover it; a stored rule's variables should become related to the caller's variables. Replacing every match by directional matching would change this relational algorithm.

`examples/chaining/tutorial/01-backward.metta`

```
!(include backward-core)
!(include graph-facts)
!(collapse (bc depth-gate 4 (reachable alice dana)))
!(collapse (bc depth-gate 3 (reachable alice dana)))
!(collapse (bc depth-gate 2 (reachable alice $where)))
!(collapse (bc depth-gate 5 (reachable dana alice)))
```

The four observations are:

1. With depth four, Alice reaches Dana with a nested `step`, `step`, `direct` proof.
2. Depth three is insufficient, so the collapsed answer is empty.
3. With depth two and an open destination, the answer is Bob with its direct proof.
4. Dana has no outgoing edge, so the last search returns no proof.

The depth counts the longest nested proof path in this implementation, including a fact leaf. It does not count all nodes in a branching proof. Each sibling receives the same remaining depth. Section 10 shows Nil's different, proof-size accounting.

On a finite knowledge base with finite premise lists and a fixed finite depth, this client has a finite search tree. Increasing the bound can expose new proofs. An empty result at one bound establishes only failure within that bound, not logical falsity.

4 Binding direction is independent of search direction

“Forward” and “backward” search concern where reasoning starts. Directional matching concerns which variables a single comparison may bind. A forward chainer and a backward chainer can each use either binding policy where appropriate.

For a pattern p and a subject s , a directional match seeks a substitution σ such that

$$\sigma(p) = s, \quad \sigma(x) = x \quad \text{for every variable of } s.$$

The subject may contain variables. Protecting them is precisely what makes this different from matching only ground data. Ordinary unification instead seeks $\sigma(p) = \sigma(s)$ and can instantiate variables on both sides.

Pattern	Subject	Directional result
(pair \$x \$x)	(pair a a)	$x \mapsto a$
(pair \$x \$x)	(pair a b)	failure
(pair \$x \$x)	(pair \$u \$u)	$x \mapsto u, u$ unchanged
(pair \$x \$x)	(pair \$u \$v)	failure for distinct u, v

The percent sign marks the *protected* side. In `pat:match%` the stored subject is protected and the query pattern binds; in `pat:%match` the query is protected and the stored patterns bind; in a `=%` rule the caller is protected and the rule head binds.

Readers who know Prolog will recognise the relations: `pat:is-instance t p` corresponds to ISO `subsumes_term(p, t)`, which likewise leaves the specific term's variables unchanged even when the two terms share variables, and `pat:is-variant` corresponds to SWI-Prolog's `=@=`.

Import one module:

```
!(import! &self pat)
```

The everyday calls keep their `pat:` prefix and return templates directly.

examples/chaining/tutorial/02-direction.metta

```
!(import! &self pat)
(edge $stored b)
(edge a b)
(edge a b)
!(collapse (match &self (edge a $y) (destination $y)))
!(collapse (pat:match% &self (edge a $y) (destination $y)))
!(collapse (pat:%match &self (edge a b) $row (covered $row)))
!(pat:let% (p $x) (p 2) (+ $x 10))
!(pat:case% (p $u) (((p a) guessed-a) ((p $x) (kept $x))))
!(pat:unify% (pair $x $x) (pair a a) (same $x) different)
!(pat:unify% (pair $x $x) (pair $u $v) (same $x) different)
!(pat:is-instance (f a a) (f $x $x))
!(pat:is-instance (f a b) (f $x $x))
!(pat:is-variant (f $x $y) (f $y $x))
!(pat:is-variant (f $x $x) (f $y $z))
```

The three stored edges include an open row and two equal ground rows. Ordinary `match` produces three destinations: it may instantiate the open stored variable. `pat:match%` produces two: only its query pattern may bind, so that open row cannot be changed into `(edge a b)`. Both copies of the ground row remain separate answers.

`pat:%match` asks the converse retrieval question: which stored patterns cover this fixed term? Its extra binder names the original stored row, so the template can report it. In this example it returns all three rows. It does not mean executing a rule backwards. The binding permission has changed; the rule's conclusion and premises have not exchanged roles.

Form	Contract
------	----------

<code>pat:match% space pattern template</code>	Retrieve with stored subjects protected; instantiate and evaluate the template for each hit.
<code>pat:%match space term output template</code>	Retrieve covering stored patterns; protect the query term and bind output to the original row (or tuple of rows for a conjunction).
<code>pat:let% pattern producer body</code>	Evaluate the producer, match each result directionally, then evaluate the body with those bindings.
<code>pat:case% producer branches</code>	Evaluate the producer and use directional branch matching under the dialect's case control contract.
<code>pat:unify% pattern subject yes no</code>	Match raw syntax, then choose the success or failure continuation.
<code>pat:is-instance term pattern</code>	Test whether the pattern can become the fixed term. Return a dialect Boolean, without exporting a substitution.
<code>pat:is-variant a b</code>	Test whether one consistent bijection of logic variable identities turns one term into the other.

The raw tests do not evaluate a call expression merely because it occurs in an operand. To compare a computed value, bind that value first with ordinary `let`. For example, the loop test in Section 8 computes `car-atom` before calling `pat:is-variant`.

Protection belongs to the matching attempt. It is not a lasting prohibition on future ordinary unification in the body. A successful match can return an open term whose variable is later bound by an explicit ordinary operation.

5 Forward chaining as controlled closure

Forward search starts from known facts and joins premises to derive more facts. Here `reach(x,y)` and `edge(y,z)` imply `reach(x,z)`. The shared variable `y` makes this a join, rather than an arbitrary pair of facts.

`examples/chaining/tutorial/03-forward.metta`

```

!(import! &self pat)
(edge alice bob)
(edge bob carol)
(edge carol dana)
(reach alice bob)
(reach bob carol)
(reach carol dana)

; This client explicitly chooses set-valued reachability. It does not merge
; proof or evidence occurrences in the general matching API.
(= (insert-new $fact)
  (if (== (collapse (pat:match% &self $fact present)) ())
    (add-atom &self $fact)
    ()))
(= (install $facts)
  (if (== $facts ())
    ()))

```

```

      (let $_ (insert-new (car-atom $facts)) (install (cdr-atom $facts))))))
(= (forward-round)
  (let $next
    (collapse (pat:match% &self
                      (, (reach $x $y) (edge $y $z)) (reach $x $z)))
    (install $next)))
(= (run-rounds $remaining)
  (if (> $remaining 0)
    (let $_ (forward-round) (run-rounds (- $remaining 1)))
    done))

!(run-rounds 1)
!(collapse (pat:match% &self (reach alice $to) $to))
!(run-rounds 1)
!(collapse (pat:match% &self (reach alice $to) $to))
!(run-rounds 1)
!(collapse (pat:match% &self (reach alice $to) $to))
!(collapse (pat:match% &self (reach dana alice) unexpected))

```

This client chooses *set-valued reachability*. Before insertion it checks whether the ground fact is already present. The matching API itself still preserves duplicates. A proof-enumerating chainer might instead keep several derivations of the same conclusion; that is a different evidence policy.

Each round first collects its candidates and then inserts them. Consequently, a new fact cannot unexpectedly become an input midway through that round. One round reaches Bob and Carol from Alice; the next also reaches Dana. A third round changes nothing. The negative query from Dana to Alice remains empty. This is a bounded, explicit closure client, not a transactional worker system or an implicit fixed-point primitive.

For a larger application, replace the repeated full scan with a delta agenda: enqueue newly admitted facts, join them with the existing index, and enqueue only new consequences. Keep the choice of duplicate policy explicit. The incremental matcher described later reuses a *query's prefix*; it is not by itself such a whole-program agenda.

6 Programmable gates without accidental guessing

A gate is an ordinary program supplied to the chainer. It can inspect a goal, the remaining budget, or a context value threaded by a richer chainer. Directional tests are useful because inspecting a goal need not refine it.

examples/chaining/tutorial/04-gates.metta

```

!(include backward-core)
!(include graph-facts)
!(import! &self pat)

; A gate can prune an application without refining the goal being inspected.
; This policy deliberately excludes a premise, so it sacrifices completeness.
(= (avoid-bc $remaining $goal)
  (if (> $remaining 0)
    (pat:unify% (edge bob carol) $goal False True)
    False))

```

```
!(collapse (bc depth-gate 4 (reachable alice dana)))
!(collapse (bc avoid-bc 4 (reachable alice dana)))
!(collapse (bc avoid-bc 2 (reachable alice bob)))
```

The first query allows the Alice–Dana proof. The second excludes the Bob–Carol premise and therefore returns no proof. The third still permits Alice–Bob. This policy intentionally sacrifices completeness relative to the unrestricted knowledge base.

The gate’s raw `pat:unify%` test does not fill an open endpoint just to make it match the forbidden edge. The post-match gate check in `bc` then rejects the forbidden edge when it has actually been selected.

Nil’s inference-control clients go further: context updates and the pruning predicate are arguments, and another chainer can prove a termination condition. The standing benchmark includes such a nested controller. The architecture does not require a new delayed-goal language to make a useful gate today. An undecided, suspended logical guard would require an additional resumption contract; this tutorial does not silently substitute a Boolean gate for it.

7 Joint constraints and the scope of protection

Some questions require several comparisons to agree on one substitution. The number of comparisons is unrestricted by the notation; “joint” does not introduce a different kind of variable for each arity.

examples/chaining/tutorial/05-joint.metta

```
!(import! &self pat)
(parent alice bob)
(parent bob carol)
(parent diane $unknown)
(parent erin frank)
!(collapse (pat:match% &self
  (, (parent $x $y) (parent $y $z)) (grandparent $x $z)))
!(pat:apply
  (pat:solve match% (((p $x) (p a))
                    ((q $x $y) (q a b))
                    ((r $y) (r b))))
  (proof $x $y))
!(collapse (pat:solve match% (((p $x) (p $subject)) ((q $x) (q a)))))
; Separate attempts do not inherit protection: this intentionally differs.
!(pat:let% (p $x) (p $subject)
  (pat:let% (q $x) (q a) (separate $subject)))
; An interactive solver can inspect an input batch whose variable names it
; did not know beforehand; a fixed application template cannot list that map.
!(pat:view (pat:solve match% (((p $x) (p a)) ((q $x $y) (q a b)))))
```

The conjunction finds exactly `(grandparent alice carol)`. The three-pair solver returns a witness that can instantiate `(proof $x $y)` to `(proof a b)`.

The next pair family is the important negative example. Its first comparison captures a subject variable; its second would have to change that protected variable into `a`. The *whole family* therefore fails. Nested `pat:let%` forms are separate attempts. The inner attempt may have a different

protected set, so the following nested example returns (separate a). Use a conjunction or `pat:solve` when all comparisons must agree under one protected set.

The advanced basis has four operations:

pat:solve Solve a list of term pairs under `match%`, `%match`, `unify` or `variant`, and return a reusable witness.

pat:apply Apply that witness to syntax. This lets a client build several outputs from one solved constraint family.

pat:view Inspect the complete variable map. An interactive constraint tool receiving arbitrary input pairs does not know their variable domain in advance; a fixed application template cannot list that domain.

pat:query Retrieve rows together with their occurrence identities and a witness. Use it when an answer's supporting rows are part of the result.

Normal template retrieval needs none of these records. A `Bindings` value is an existing grounded substitution value, not a rule to interpret a user-created expression with that printed name. Treat the witness as an opaque value and use its operations.

For the solved pair family above, `pat:view` returns:

```
(Bindings (($x a) ($y b)) ())
```

The first field lists assignments; the second retains residual equality constraints. The final `()` means that none remain. Dropping that field would hide obligations in a result that has residual constraints. This is the existing bindings representation; ordinary template queries still return their templates directly.

For a joint directional solve, subject variables from the entire family are protected. Incremental execution must also notice a variable that becomes a protected subject only in a later pair. The native prefix state checks that case and refuses to revise an earlier image to force success.

8 Loop tests, memoization and tabling

Two goals are variants when they have the same shape and the same variable sharing, up to a one-to-one renaming. `(f $x $x)` and `(f $u $u)` are variants; `(f $x $x)` and `(f $u $v)` are not. This concerns free logic variables. It is distinct from alpha-equivalence of bound variables in a higher-order language.

examples/chaining/tutorial/07-loop-tests.metta

```
!(import! &self pat)
(= (seen-variant $goal $ancestors)
  (if (== $ancestors ())
      False
      (let $ancestor (car-atom $ancestors)
        (if (pat:is-variant $goal $ancestor)
            True
            (seen-variant $goal (cdr-atom $ancestors))))))
```

```

!(seen-variant (reachable alice $where) ((reachable alice $old)))
!(seen-variant (reachable alice bob) ((reachable alice $old)))
!(seen-variant (pair $x $x) ((pair $a $b)))
!(pat:is-instance (reachable alice bob) (reachable alice $old))

; A more general ancestor covers this goal syntactically. That alone does
; not show that the ancestor's answers have already been computed.
; Pruning recursive calls and completing a table are different algorithms.

```

The answers are true, false, false, true. A ground destination is an instance of an open destination, but they are not variants. Computing the ancestor first is essential because the relation test inspects raw syntax.

For terms that share variable identities, protection also matters. The terms `(f $x $y)` and `(f $y $x)` are variants. Neither can be turned into the other by a substitution that fixes all variables of the specific operand. Thus do not implement this protected instance predicate by assuming that two independently tested directions always reproduce the variant predicate.

An ancestor check can prevent repeating a goal along one branch. It does not show that all answers to that goal have been found. In particular, treating an unfinished recursive call as a completed empty table can lose proofs. Tabling manages dependencies and answer completion; memoization reuses completed calls according to its cache contract.

<examples/chaining/tutorial/08-tables.petta.metta>

```

!(import! &self (library lib_tabling))
!(tabled (fib $n))
(= (fib 0) 0)
(= (fib 1) 1)
(= (fib $n)
  (if (> $n 1) (+ (fib (- $n 1)) (fib (- $n 2))) (empty)))
!(fib 20)

; Tabling deliberately coalesces equal answers; ordinary equations do not.
!(tabled (tabled-proof $goal))
(= (tabled-proof goal) proof)
(= (tabled-proof goal) proof)
(= (ordinary-proof goal) proof)
(= (ordinary-proof goal) proof)
!(collapse (tabled-proof goal))
!(collapse (ordinary-proof goal))
!(collapse (tabled-proof missing))

!(import! &self (library lib_memo))
(= (square $x) (* $x $x))
!(memoize square)
!(square 9)
!(square 9)

```

This chapter uses the existing PeTTa libraries and works in plain PeTTa as well as extended. `fib 20` returns 6765. The tabled duplicate proof has one answer, while the ordinary duplicate equations have two. The missing goal has no answer. Both memoized square calls return 81.

Coalescing table answers may be right for reachability and wrong for an evidence bag. If different derivations must remain distinguishable, include their proof or evidence identity in the tabled answer and choose the intended semantics deliberately. Cache reuse for effectful or changing computations also requires the library's invalidation and effect contract; the square example is intentionally pure.

9 Evidence you can inspect

examples/chaining/tutorial/06-evidence.metta

```
!(import! &self pat)
(evidence sensor-a (edge alice bob))
(evidence sensor-a (edge alice bob))
(evidence sensor-b (edge bob carol))

; Equal data can be distinct observations. Keep the supporting occurrences
; until an explicit evidence policy decides how they should be combined.
!(collapse
  (let (pat:hit $rows $w)
    (pat:query &self match%
      (, (evidence $first (edge $x $y))
        (evidence $second (edge $y $z))))
    (derivation (pat:apply $w (reachable $x $z)) $rows)))

; A missing premise does not create an evidence certificate.
!(collapse (pat:query &self match% (evidence $source (edge carol dana))))
```

The first query returns two derivations of `(reachable alice carol)`. Their first supporting rows have different occurrence identities, although their labels and values are equal. Their second supporting row has the same identity. The last query returns no certificate.

The record vocabulary is small:

```
(pat:hit rows witness)
(pat:row occurrence original instantiated)
(pat:occurrence space-id revision row-id)
```

Rows appear in premise order. The identity refers to a row in a particular space read, not a universal persistent evidence name. A program that needs durable source identity should also store its own source labels and metadata. The original row retains its pattern; the instantiated view shows the selected match. Equal values can have different occurrences.

These identities make dependence visible. They do *not* establish that two observations are statistically independent, and they do not automatically prevent double counting. In this example, both derivations share the second observation. Any probabilistic revision must account for that overlap using its own evidence model.

9.1 Forward and backward clause subsumption

For finite first-order clauses, C subsumes D when one substitution on C makes every literal of C occur in D , preserving polarity. The choice of literal correspondences is part of the search. It is

more than testing each literal independently.

examples/pattern/clause_retrieval.metta

```
!(import! &self pat:clause)
(clause broad ((pos (P $x))))
(clause linked ((pos (P $x)) (pos (Q $x))))
(clause linked ((pos (P $x)) (pos (Q $x))))
(clause crossed ((pos (P a)) (pos (Q b))))
(clause negative ((neg (P $x))))

; Forward: which existing clauses cover the new clause?
; The occurrence identifies the supporting evidence: the two identical linked
; clauses remain distinct. A template containing only their values loses this
; provenance. The clause witness is reusable on its original literal syntax.
(= (forward $kb $new)
  (let (pat:hit ((pat:row $occ (clause $name $old) $view)) $row-witness)
    (pat:query $kb match% (clause $label $body))
    (let $w (pat:subsume set $old $new)
      (cover $occ $name $old (pat:apply $w $old)))))

; Backward: which existing clauses does the new clause cover?
(= (backward $kb $new)
  (let (pat:hit ((pat:row $occ (clause $name $old) $view)) $row-witness)
    (pat:query $kb match% (clause $label $body))
    (let $w (pat:subsume set $new $old)
      (cover $occ $name $old (pat:apply $w $new)))))

; Literal order is irrelevant, but one substitution must cover every literal.
!(collapse (forward &self ((pos (Q a)) (pos (P a)))))
!(collapse (backward &self ((pos (P $candidate)))))
; Occurrence-sensitive coverage is an explicit different policy.
!(collapse
  (let $w (pat:subsume set ((pos (P $x)) (pos (P $y))) ((pos (P a))))
    (pat:apply $w (covered $x $y)))
  !(collapse
    (pat:subsume multiset ((pos (P $x)) (pos (P $y))) ((pos (P a)))))
  ; Finding logical redundancy does not remove evidence occurrences.
  !(collapse (match &self (clause $name $body) $name)))
```

Forward subsumption asks which old clauses cover the new one. Backward subsumption asks which old clauses the new one covers. The optional `pat:clause` module performs the clause search; it is not a module of short aliases. The example retains two equal stored clauses as two evidence occurrences even when both prove the same logical redundancy.

Set and multiset clauses have different contracts. Set coverage may reuse a literal of the specific clause. Multiset coverage requires separate occurrences. The positive set example and negative multiset example in this file make that difference observable. A theorem about redundancy does not itself authorize deleting evidence or changing a proof bag.

9.2 PLN: choose premises before combining truth values

Directional retrieval can select the premise topology of induction or abduction while leaving evidence untouched. Induction often compares two links with a common source; abduction com-

pares links with a common target. Those topologies do not turn matching itself into probabilistic inference.

The maintained client `examples/pattern/pln_selection.metta` selects evidence and priors and builds a value named `selected`. Its dialect wrappers import their existing PLN truth functions:

```
./cetta --lang he --profile extended examples/pattern/pln_selection.he.metta
./cetta --lang petta --profile extended examples/pattern/pln_selection.petta.metta
```

The example includes an open fact that must not be instantiated to manufacture a missing premise, and duplicate observations that remain duplicate selections. It reports the truth-function inputs alongside the conclusion. The current HE and PeTTa libraries use different confidence formulas; the client preserves each library’s result rather than claiming cross-dialect formula identity.

Use `pat:query` instead of value-only retrieval when a revision policy needs occurrence identities. Keep separate the questions “does this tuple fit the inference schema?” and “what evidence model justifies combining its truth values?”

10 Nil’s proof-size search and backward-via-forward search

Nil’s optimized propositional chainer carries (`MkSized size proof`). An axiom costs one node. A modus-ponens proof combines an implication proof of size f , an antecedent proof of size x , and one application node:

$$\text{size}(\text{mp}(F, X)) = f + x + 1.$$

For a total allowance s , the implementation first reserves room for the application and the second proof, then passes the actual remainder $s - 1 - f$ to the second recursive call. This is not the depth bound from our first chainer.

`examples/chaining/tutorial/09-nil-size.petta.metta`

```
; Use the pinned upstream engine already maintained by the chainer ladder.
!(include ../../../../benchmarks/chaining/nil_current/engines/obc)
!(collapse (obc 0 (: $proof (→ A (→ B A)))))
!(collapse (obc 1 (: $proof (→ A (→ B A)))))
!(collapse (obc 4 (: $proof (→ A A)))
!(collapse (obc 5 (: $proof (→ A A)))
```

The included engine is the pinned source already used by the chainer ladder. Zero budget rejects an axiom; one admits it. Four nodes cannot prove the identity formula in this search; five produce

```
(MkSized 5 (: (mp (mp ax2 ax1) ax1) (→ A A)))
```

The bound counts proof nodes, not runtime instructions, memory or elapsed time. A program may expose any of these accounts, but they are different quantities.

10.1 Changing the representation of unfinished work

Backward search holds unresolved premises on its call stack or agenda. Nil's backward-via-forward experiments instead build proof contexts carrying pending hypotheses and progressively discharge them. Both can target the same theorem while exploring different intermediate objects.

The maintained semantic-closure examples are:

```
./cetta --lang he examples/chaining/obfc_xp_he.metta  
./cetta --lang petta examples/chaining/obfc_xp_petta.metta
```

Their state carries a depth allowance, a count of pending hypotheses, a target and a partial proof context. The count gives an immediate necessary condition: if more hypotheses remain than the allowance can discharge, that branch cannot succeed under the bound. It is a control invariant, not permission to invent a missing premise.

These examples preserve the semantic search relation of the stated MM2 experiment. Its particular MORK join plan, including the order of its decrement relation, remains a separate physical execution strategy. The examples do not claim that CeTTa executes that MM2 plan unchanged.

10.2 A practical combination

Use ordinary unification to discover proof variables and connect premises. Use directional tests to inspect open goals and select evidence without changing it. Thread an explicit proof budget and a programmable gate. Return proof data and, where needed, occurrence identities. Then choose a table or agenda whose duplicate policy agrees with the intended answers.

That combination supports a relational proof generator, a cautious controller, and an inspectable result without adding a special primitive for every chaining strategy.

11 Rules that rewrite without narrowing

Ordinary = rules may instantiate a caller during head unification. An extended-profile =% rule instead matches its head against a protected caller. Rejected heads do not evaluate their bodies. The policy is attached to the rule; ordinary and directional rules can coexist.

examples/pattern/ski.he.metta

```
; HE supports a compound operator. Each declaration is a native rule.  
(=% (I $x) $x)  
(=% ((K $x) $y) $x)  
(=% (((S $x) $y) $z) (($x $z) ($y $z)))  
(=% ((S K) K) I)  
  
!(((S K) K) a)  
!((K a) b)  
; This call is too general: its variable must not be manufactured into K.  
!((S K) $unknown)  
  
; Ordinary relational equations can refine a caller when that is intended.  
(= (relational K) selected)
```

```
!(let $answer (relational $caller) (answer $answer $caller))
```

The short rule $((S\ K)\ K)$ can reduce to I . It must not fire on $((S\ K)\ \$unknown)$ by inventing $\$unknown = K$. That partial application stays unreduced in the HE example. The ordinary relational rule at the end intentionally does refine its caller.

PeTTa has a different compound-operator contract. Its companion makes the syntax being reduced an explicit argument:

examples/pattern/ski.petta.metta

```
; PeTTa evaluates compound operators differently from HE. A named function
; exposes a reduction step over SKI syntax without changing that contract.
(=% (ski (I $x)) $x)
(=% (ski ((K $x) $y)) $x)
(=% (ski (((S $x) $y) $z)) (($x $z) ($y $z)))
(=% (ski ((S K) K)) I)

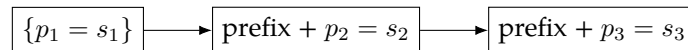
!(ski ((S K) K))
!(ski ((K a) b))
; A known PeTTa function with no applicable clause has no answer.
!(collapse (ski ((S K) $unknown)))
(= (relational K) selected)
!(let $answer (relational $caller) (answer $answer $caller))
```

Here the successful reductions return I and a . A known PeTTa function with no applicable clause has no answer; the collapsed partial case is empty. This is one reason to publish both dialect examples.

Head protection does not freeze a variable for the entire future computation. The body of a successful $\text{=}\%$ rule can still call ordinary unification. For purely syntactic rewriting, keep that body within operations whose contracts preserve the intended input syntax.

12 Costs and limits

Joint matching now retains a successful prefix and extends it with the next premise. It keeps the earlier images and protected identities instead of re-matching every earlier pair for every candidate.



one consistent substitution
with retained protection

This removes repeated term traversal. It does not eliminate the number of candidate combinations or the size of the requested answer bag. Current prefix snapshots also copy binding and occurrence metadata, so arbitrarily wide joins are not promised constant work per added premise. In HE a joint query computes its complete finite set of answers before returning them; PeTTa can return answers one at a time.

The checked model concerns first-order matching with a declared equality for leaves. Binder-aware higher-order matching needs a separate treatment of scope, equality, search and resources. No higher-order completeness claim follows from these tests. Machine-checked Lean proofs establish the laws of the mathematical model; they are not needed at run time, and they do not verify the compiled C program as a whole.

An interrupted search is not a completed empty search. Check the process and completion outcome when running bounded experiments; do not feed an unfinished frontier into a proof or evidence consumer as if it were complete.

Appendix B lists the commands that check these claims and records one measurement of the prefix reuse.

13 Exercises with counterexamples

1. Add `(fact da (edge dana alice))`. Compare bounded answers at depth four and eight. Which new proofs repeat a vertex? Does rejecting such proofs preserve all paths, all reachable pairs, or neither?
2. Duplicate `(fact ab (edge alice bob))`. Predict the change in proof multiplicity. Then compare it with the forward client's set-valued closure. Explain why the different counts are intentional.
3. Add `(parent carol $unknown)` to the joint example. Can a later premise turn that protected variable into a named parent? Construct a positive shared-variable match and a negative inconsistent one.
4. Replace the raw ancestor test's bound `$ancestor` with the unevaluated expression `(car-atom $ancestors)`. Explain why that tests different syntax and fails to recognize the intended variant.
5. In the evidence example, make both derivations use one observation. Show that two proof trees do not imply two independent observations.
6. Compare a proof-height budget with Nil's total proof-size budget on a branching rule. Give a proof admitted by one numeric bound and excluded by the other when both bounds have the same numeral.
7. Build a small agenda for forward closure. State its duplicate policy and show that changing the queue order preserves its intended finite closure. Keep answer order separate from set equality.

A Reference card

All operations below require an extended profile and, except for `=%`, one `!(import! &self pat)`.

Form	What may bind
<code>(= head body)</code>	ordinary rule: the head and the caller may both bind
<code>(=% head body)</code>	directional rule: only the head binds; the caller is protected
<code>(match space pattern template)</code>	ordinary retrieval: both sides may bind

(pat:match% space pattern template)	only the pattern binds; stored rows are protected
(pat:%match space term \$row template)	only stored rows bind; \$row names each stored row that covers term
(pat:let% pattern producer body)	evaluate producer; bind only pattern
(pat:case% producer branches)	evaluate producer; each branch pattern binds, the value is protected
(pat:unify% pattern subject yes no)	raw syntax; only pattern binds
(pat:is-instance term pattern)	test only; ISO subsumes_term(pattern, term)
(pat:is-variant a b)	test only; one renaming of variables, SWI =@=
(pat:solve policy pairs)	one witness for several pairs at once; policy is match%, %match, unify or variant
(pat:apply witness syntax)	instantiate syntax with a witness
(pat:view witness)	inspect assignments and residual equality constraints
(pat:query space policy pattern)	rows with their occurrence identities and a witness

With the rows (edge \$stored b), (edge a b) and (edge a b): match of (edge a \$y) gives three answers, pat:match% gives two, and pat:%match of (edge a b) returns all three stored rows.

Separate pat:let% forms are separate attempts: an inner attempt does not inherit the outer one's protected variables. Use a conjunction pattern or pat:solve when several comparisons must share one protected set.

B Reproducing the checks and measurements

The maintained checks are:

```
make BUILD=python test-pattern test-chaining-tutorial
make BUILD=python test-petta-upstream-profiles PETTA_ORACLE_ROOT=../PeTTa
make BUILD=python test
```

The pattern gate includes finite-model correspondence, repeated-variable and alias tests, allocation failures, deliberate defects, and interruption controls. The tutorial gate independently checks every row of a larger 12-premise proof join, including its duplicate-occurrence combinations.

To collect a scaling record with instruction counts:

```
make BUILD=python ENABLE_RUNTIME_STATS=1
python3 scripts/bench_pattern.py --cetta ./cetta \
  --stats-cetta runtime/cetta-python-runtime-stats \
  --reference-cetta path/to/pinned-cetta --samples 5 --instructions \
  --output results/pattern-comparison
```

Keep a reference executable in its matching repository context so native libraries resolve correctly. The driver checks answers before accepting a cost record, identifies its sources and binaries, and alternates execution order. Compare both same-profile versions and plain versus extended: they answer different performance questions. A measured finite ladder is evidence about those clients, not a universal worst-case percentage guarantee.

B.1 A measured example

On the qualified GCC 15.2 Python build, five interleaved samples of the fixed-size prefix workload gave the following median user-instruction counts (millions). The reference is the directional implementation before prefix reuse; both executables return the same checked answers.

Premises	HE reference	HE current	PeTTa reference	PeTTa current
2	26.508	26.181	12.492	12.352
8	34.113	32.024	12.733	12.448
32	37.970	33.045	16.902	13.611
64	48.642	34.428	28.163	15.228

At 64 premises this is about 29% fewer instructions for HE and 46% for PeTTa. The mechanism counters grow by five subject views and six pair visits per additional premise between the wider rows, after fixed setup work. The benchmark driver rejects renewed quadratic prefix traversal.

A separate eleven-client chainer ladder uses five balanced samples to compare each profile against the original pre-directional baseline, and extended directly against plain. The largest plain-PeTTa median instruction increase is 0.14%; extended uses between 0.75% and 16.93% fewer instructions than its baseline. Extended's largest remaining instruction difference from plain is 0.94% across this ladder, with identical checked outputs.

For example, backward Loowoz at size 19 uses 32.90 billion instructions in extended, down from 39.61 billion; the current plain profile uses 32.74 billion. Compiled calls reuse the ordinary fast path while their definitions remain unchanged, retaining the existing fallback when declarations change. Compiler, input shape and machine changes require fresh measurements.

Sources and companion programs

The small Horn and closure clients in this guide are explanatory programs. The larger Nil clients are maintained with pinned provenance in `benchmarks/chaining/nil_current/source_provenance.tsv` and their source headers. The proof-size engine records revision `2151416f4d44b3b2dbbcd57a0dd7ffd5a75c992c`; the MM2 semantic-closure examples record `9a3287210880ca162beed379942a036a6f3cf8a6`.

- Nil Geisweiller's chaining experiments: <https://github.com/ngeiswei/chaining>.
- PeTTa's proof-producing nilbc example: <https://github.com/trueagi-io/PeTTa/blob/main/examples/nilbc.metta>.
- The one-sided matching request and the percent convention: <https://github.com/trueagi-io/hyperon-experimental/issues/674>.
- Executable directional clients and their expected observations: `examples/pattern/`.
- Reproduction drivers: `scripts/bench_chaining.py`, `scripts/bench_pattern.py`, and `tests/suport/check_chaining_tutorial.py`.